SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

*I **Dhaanush Rajan(192421162)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANSWER

1) TEST CASE DESIGN FOR ONLINE BANKING FUND TRANSFER SYSTEM

1. Introduction and Problem Analysis

An online banking system is experiencing frequent failures during fund transfers, particularly at boundary values such as the minimum and maximum permitted transfer amounts. This may indicate defects in input validation, limit enforcement, transaction processing, database updates, or error handling.

The objective is to evaluate the existing test-case design, apply suitable black-box techniques, assess random testing, and propose a structured approach that improves defect detection.

2. Critical Evaluation of the Current Testing Strategy

Identified issue	Possible consequence	Recommended improvement
Insufficient boundary testing	Failures at minimum and maximum limits may remain undetected.	Apply Boundary Value Analysis (BVA).
Inadequate input classification	Invalid values may be accepted or valid values rejected.	Apply Equivalence Partitioning (EP).
Excessive dependence on random testing	Critical edge cases may not be selected systematically.	Use deterministic, risk-based test selection.
Missing transaction-state checks	Balances may be updated incorrectly or transfers duplicated.	Test atomicity, idempotency, and rollback.
Incomplete negative testing	Invalid or unauthorized requests may be processed.	Test zero, negative, malformed, and excessive amounts.
Unclear expected outcomes	Defects may go unnoticed or tests may be inconclusive.	Define explicit pass/fail criteria for every test.

Critical assessment: The main weakness is not necessarily the number of test cases, but whether the selected inputs cover valid, invalid, and high-risk conditions in a systematic way.

3. Equivalence Partitioning (EP)

Equivalence Partitioning divides the input domain into groups whose values are expected to behave similarly. A representative value is selected from each group, reducing redundant tests while retaining meaningful coverage.

Assumption for this example: the permitted transfer amount is ₹100 to ₹50,000 inclusive. Other conditions—such as available balance, authentication, beneficiary validity, daily limits, and account restrictions—must also be satisfied.

Partition	Input class	Representative input	Expected result
EP01	Valid amount: ₹100–₹50,000 inclusive	₹10,000	Accept if all other conditions pass.
EP02	Below minimum	₹50	Reject with a validation message.
EP03	Above maximum	₹50,001	Reject with a validation message.
EP04	Zero or negative amount	₹0 and −₹100	Reject.
EP05	Blank amount	Empty input	Reject and identify the missing value.
EP06	Non-numeric input	ABC	Reject with a format error.
EP07	Unsupported decimal	₹100.123	Reject or normalize according to

	precision		the documented currency rules.

Justification: EP covers the principal classes of valid and invalid values without testing every possible amount. The precise treatment of decimals and formatting must be based on the system requirements.

4. Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on values immediately below, at, and immediately above the edges of a permitted range. It is especially appropriate because the reported failures occur near the transfer limits.

Test ID	Transfer amount	Boundary condition	Expected result
BVA01	₹99	Minimum − 1	Reject.
BVA02	₹100	Minimum	Accept if all other checks pass.
BVA03	₹101	Minimum + 1	Accept if all other checks pass.
BVA04	₹49,999	Maximum − 1	Accept if all other checks pass.
BVA05	₹50,000	Maximum	Accept if all other checks pass.
BVA06	₹50,001	Maximum + 1	Reject.

These tests can expose off-by-one defects and incorrect comparison operators. For example, using amount > 100 instead of amount >= 100 would incorrectly reject the permitted minimum of ₹100. Similarly, an incorrect upper-limit condition might allow ₹50,001.

Note: A valid amount is not sufficient by itself to guarantee a successful transfer. Expected success also depends on available funds, authentication, beneficiary details, daily limits, and other business rules.

5. Additional Critical Test Scenarios

Test ID	Scenario	Expected result
TC01	Transfer exactly the available balance	Success if permitted by account rules; final balance is correct.
TC02	Transfer more than available balance	Reject; no debit or credit occurs.
TC03	Transfer to an invalid beneficiary	Reject; no debit occurs.
TC04	Transfer without valid authentication	Reject; account remains unchanged.
TC05	Submit the same transfer twice	Prevent duplicate debit using idempotency controls.
TC06	Network failure during transfer	No inconsistent partial transfer; transaction state can be safely resolved.
TC07	Exceed the daily transfer limit	Reject according to configured limits.
TC08	Retry after a timeout	Return the original result or safely resolve the transaction without duplication.
TC09	Two simultaneous transfers from the same account	Prevent overspending and maintain balance consistency.
TC10	Database failure during processing	Roll back incomplete operations or recover safely.

These scenarios extend black-box testing to business rules and transaction behavior. They are important because a transfer can pass amount validation and still fail through concurrency, recovery, or integration defects.

6. Evaluation of Random Testing

Random testing generates inputs randomly from a chosen domain. It can reveal unexpected failures and unusual combinations, but it does not guarantee that important boundary values will be tested.

Advantages	Limitations
Can discover unexpected defects.	May rarely select exact boundary values.
Useful for large input spaces.	Does not guarantee systematic coverage.
Can reveal unusual input combinations.	Failures can be hard to reproduce unless inputs and seeds are recorded.
Useful for robustness and stress testing.	Generated cases may be redundant or low-risk.

Should random testing be retained?

Recommendation: retain random testing as a supplementary technique, not as the primary or only strategy. EP and BVA should provide systematic coverage, while random testing explores additional values and combinations.

1) Use Equivalence Partitioning to cover representative valid and invalid input classes.
2) Use Boundary Value Analysis to explicitly test values below, at, and above each limit.
3) Use random testing to explore unexpected combinations and unusual values.
4) Use property-based testing to check general invariants across generated transactions.
5) Use regression testing to repeat previously failing cases after fixes.

Random tests should record their inputs and random seed so failures can be reproduced. For a randomly selected integer between ₹1 and ₹100,000, any one exact boundary value may be unlikely to appear in a small test run.

7. Structured and Optimized Testing Approach

Step 1 — Requirement analysis: Confirm transfer limits, currency precision, balance rules, daily restrictions, authentication requirements, and error-handling expectations.
Step 2 — Design test partitions: Separate valid, invalid, malformed, and restricted input classes using EP.
Step 3 — Execute boundary tests: Test minimum − 1, minimum, minimum + 1, maximum − 1, maximum, and maximum + 1.
Step 4 — Validate transaction integrity: Test balance consistency, duplicate requests, concurrency, failures, and rollback.
Step 5 — Supplement with generated tests: Use random and property-based testing to explore additional inputs and verify invariants.
Step 6 — Regression and release decision: Re-run critical tests, review coverage and defect results, and ensure no unresolved critical defects remain.

Optimization measures

1. Prioritize high-risk cases involving transfer limits, balances, duplicate requests, and transaction failures.
2. Automate repeatable API-level tests for boundary values and negative scenarios.
3. Use data-driven testing by storing test inputs and expected outcomes in a structured dataset.

4. Verify financial invariants: for a successful transfer of amount A, the sender's balance decreases by A and the recipient's balance increases by A, adjusted for any applicable fees; ledger records must remain consistent.
5. Measure boundary coverage, requirements coverage, defect detection rate, regression pass rate, and escaped defects.
6. Use a dedicated test environment and test accounts; do not run destructive or concurrent tests against real customer accounts.

8. Sample Automated Test Logic

The following Python example demonstrates validation of integer transfer amounts for the assumed limits. It tests the amount range only and is not a complete banking transaction implementation.

```
MIN_TRANSFER = 100
MAX_TRANSFER = 50000

def validate_transfer(amount):
    if not isinstance(amount, int) or isinstance(amount, bool):
        return False
    return MIN_TRANSFER <= amount <= MAX_TRANSFER

test_cases = [
    (99, False),
    (100, True),
    (101, True),
    (49999, True),
    (50000, True),
    (50001, False),
    (0, False),
    (-100, False),
]

for amount, expected in test_cases:
    actual = validate_transfer(amount)
    status = "PASS" if actual == expected else "FAIL"
    print(
        f"{status}: amount={amount}, "
        f"expected={expected}, actual={actual}"
    )
```

Production systems must also handle decimal currency values correctly and validate authentication, available balance, beneficiary details, applicable limits, and atomic transaction processing. Validation must be enforced server-side, not only in the user interface.

9. Expected Outcomes and Effectiveness

Technique	Primary contribution
Equivalence Partitioning	Reduces redundant cases while covering meaningful input classes.
Boundary Value Analysis	Detects edge-condition and off-by-one defects.
Negative testing	Verifies rejection of invalid or unauthorized requests.
Random/property-based testing	Finds unexpected combinations and invariant violations.
Regression testing	Prevents previously fixed defects from returning.
Concurrency and recovery testing	Detects duplicate, partial, and inconsistent transfers.

No testing strategy can guarantee detection of every defect. Effectiveness should be judged by measurable coverage, defect discovery, and transaction-integrity results—not simply by the number of tests executed.

10. Conclusion

The frequent fund-transfer failures suggest that the existing test design may not adequately address boundary conditions and critical transaction rules. Equivalence Partitioning should classify valid and invalid inputs, while Boundary Value Analysis should explicitly test values immediately below, at, and above the minimum and maximum limits.

Random testing should be retained because it can reveal unexpected defects, but it should not replace systematic black-box testing. The recommended strategy combines EP, BVA, negative testing, transaction-integrity checks, random or property-based testing, and automated regression testing. This approach improves coverage, makes failures reproducible, reduces redundant execution, and provides stronger assurance that transfers are processed accurately and securely.

2)Healthcare Application: White-Box Coverage and Control Flow Graph

1. Problem and coverage gaps

Five critical post-deployment defects indicate that test adequacy was not established solely by counting executed tests. Possible gaps include untested decision outcomes, omitted error-handling paths, untested input combinations, and missing boundary or exception cases. The exact defects and code are not supplied, so the following control-flow graph (CFG) is conceptual.

2. Conceptual CFG

Node	Operation	Possible next node
1	Start and receive patient data	2
2	Validate input and patient identifier	3 if valid; 8 if invalid
3	Check required measurements	4 if present; 8 if missing
4	Apply clinical calculation / rule	5
5	Check risk threshold	6 if high risk; 7 otherwise
6	Raise alert and record escalation	9
7	Store normal result and display outcome	9
8	Return validation/error message; log error	9
9	End	—

This model illustrates input validation, a missing-data branch, a clinical threshold decision, normal processing, and error handling. Actual nodes and edges must be derived from the implementation.

3. Coverage analysis

1. Statement coverage: every executable statement runs at least once. It can miss an untested false outcome of a decision.
2. Branch coverage: every decision outcome (true and false) executes at least once. It is stronger than statement coverage but does not guarantee every combination of conditions.
3. Path coverage: each possible path executes. Complete path coverage is often infeasible when loops create infinitely many paths, so critical and feasible paths must be prioritized.
4. Condition coverage: each Boolean condition is shown to evaluate both true and false, where applicable.

Example missing paths include invalid patient ID, missing measurement, high-risk result, normal result, and error handling. A suite that only tests valid patients with normal measurements can execute many statements while missing clinically critical branches.

4. Improved strategy

1. Derive a CFG from the source code and identify decisions, loops, exception edges, and exits.
2. Create tests for each branch outcome and each validation/error path.
3. Use boundary tests around clinical thresholds and valid measurement ranges, using approved requirements rather than invented medical cutoffs.
4. Use basis-path testing and cyclomatic complexity to identify a feasible set of linearly independent paths.
5. Add integration tests for data storage, alert delivery, and audit logging.
6. Run risk-based regression tests and review coverage reports; investigate every uncovered safety-critical branch.

This approach improves reliability by testing both normal and exceptional processing. Coverage is evidence of exercised structure, not proof of clinical correctness; expected results must also be validated by domain experts.

3)E-Commerce Price, Quantity and Discount Combinations

1. Limitations of current testing

Testing price, quantity, and discount independently can miss interaction defects—for example, a discount applied twice, a negative total after a percentage discount, or rounding errors when quantity is multiplied by price. Exhaustively testing every value may be impractical, so the design should combine EP, BVA, decision tables, and pairwise or risk-based combination coverage.

2. Equivalence classes and boundaries

Field	Valid classes	Invalid classes / boundaries
Price	Positive supported currency amount	Zero, negative, malformed; minimum permitted price and adjacent values.
Quantity	Positive integer within stock/order limit	Zero, negative, fractional if not allowed, maximum + 1.
Discount	Allowed percentage or fixed amount	Negative, above 100% for percentage discounts, greater than subtotal, malformed.

Exact limits and discount rules must be taken from requirements. BVA should test each numeric lower/upper limit at limit − 1 unit, the limit, and limit + 1 unit where that unit is meaningful.

3. Illustrative decision table

Rule	Valid item/price	Quantity available	Discount valid	Expected result
R1	Yes	Yes	Yes	Calculate total; accept if total is non-negative.
R2	No	—	—	Reject invalid price/item.
R3	Yes	No	—	Reject quantity; do not place order.
R4	Yes	Yes	No	Reject discount or apply documented fallback.
R5	Yes	Yes	Yes, discount > subtotal	Reject or cap only if explicitly specified; total must not become negative.

4. Example test cases

- Minimum valid price, quantity 1, no discount: total equals unit price.
- Maximum supported quantity with valid stock: total and stock reduction are correct.
- Quantity 0 or negative: reject without order creation.
- Discount 0%, exactly the allowed maximum, and maximum + a small unit: verify documented rule.
- Discount equals subtotal: total is zero only if allowed by business rules.
- Discount exceeds subtotal: no negative total unless the business explicitly supports credit balances.
- Combine high price, maximum quantity, and discount to check overflow, rounding, and arithmetic order.

5. Revised approach and justification

Use EP to choose representative values, BVA for limits, and a decision table for interactions. Add pairwise coverage for broad combinations and exhaustive coverage for high-risk rules. Validate totals against an independent oracle and check invariants such as total ≥ 0, discount applied once, and stock never below zero. This detects interaction defects without blindly enumerating every possible input.

4)Complex Nested Conditions and Loops: Path Testing

1. Evaluation

Partial path coverage in a module with nested decisions and loops can leave combinations of conditions, loop exits, and exception paths untested. The team should not claim complete path coverage merely because statements or some branches executed.

2. CFG construction and independent paths

Construct a directed graph in which nodes represent statements or basic blocks and edges represent possible control transfers. Include true/false decision edges, loop-back edges, loop exits, exception handling, and

early returns. Identify independent paths—each adds at least one edge not already represented by previously selected paths.

Cyclomatic complexity estimates the number of linearly independent paths: $V(G) = E - N + 2P$, where E is the number of edges, N is the number of nodes, and P is the number of connected components (normally 1 for a single routine). It can also be calculated as the number of predicate nodes + 1 for a simple connected CFG.

3. Illustrative nested-loop model

Path	Scenario	Expected observation
P1	Outer loop has zero iterations	Immediate correct exit.
P2	Outer loop executes once; inner loop zero times	No inner processing; normal exit.
P3	Both loops execute once	One expected processing operation.
P4	Inner loop executes multiple times	Correct repeated processing and termination.
P5	Outer loop reaches its termination boundary	Correct exit without an extra iteration.
P6	Invalid data or exception inside loop	Safe handling and no corrupted state.

These are conceptual paths; actual paths must be selected from the module's CFG. Exhaustive path testing is usually impossible with loops, so basis paths and risk-critical paths are selected.

4. Strategy

- Draw and review the CFG against source code.
- Calculate cyclomatic complexity and select a basis set of independent paths.
- Exercise true and false outcomes of each decision and relevant nested-condition combinations.
- Test loop counts of zero, one, two, typical, maximum supported, and invalid/unbounded inputs.
- Use timeouts and watchdogs to detect non-termination.
- Use instrumentation and regression automation to verify coverage and prevent recurrence.

This strategy exposes missing control-flow behavior and loop exit defects. Coverage must be combined with assertions about outputs and state.

5)Financial Application: Hybrid Black-Box and White-Box Testing

1. Why defects still escape

Using both black-box and white-box testing is not sufficient unless test cases are derived from requirements and code structure, coverage criteria are appropriate, and integration and recovery behavior are tested. Possible weaknesses include boundary omissions, untested internal branches, incorrect assumptions in expected results, concurrency gaps, and poor regression discipline.

2. Hybrid test design

Layer	Techniques	Example
Requirement/input layer	EP, BVA, decision tables	Test amount below/at/above transfer limit and combinations with balance or daily limit.
Code structure layer	CFG, statement/branch/condition and basis-path testing	Exercise each validation branch, successful posting, rollback, and exception route.
Integration layer	API/database/service tests	Confirm debit, credit, ledger, audit, and notifications agree.
Reliability layer	Concurrency, fault injection, idempotency, regression	Retry after timeout and simultaneous transfers; verify no duplicate debit.

3. Coverage and adequacy

- Statement coverage confirms executed statements, but not all decisions.
- Branch coverage exercises each decision outcome, but may miss combinations of conditions.
- Condition coverage checks each Boolean condition as true and false.
- Basis-path testing targets independent paths using CFG and cyclomatic complexity.
- Requirements traceability links each business rule to test cases and results.

4. Example hybrid test set

1. Test transfer amount at minimum − 1, minimum, minimum + 1, maximum − 1, maximum, and maximum + 1.
2. Test sufficient and insufficient balance, valid and invalid beneficiary, and daily-limit conditions using a decision table.
3. Use CFG analysis to cover accepted, rejected, rollback, and exception branches.
4. Inject a failure after debit but before credit and verify atomicity or safe compensation.
5. Repeat a request with the same idempotency key and verify only one transfer is posted.
6. Automate regression tests for all previously discovered defects.

Conclusion: Hybrid testing provides complementary evidence—black-box methods verify externally visible requirements while white-box methods expose unexecuted code paths. Integration and risk-based tests address defects that neither technique alone guarantees to find.

6)HR Payroll Bonus Errors at 5, 10 and 15 Years

1. Analysis

Incorrect payouts exactly at 5, 10, and 15 years strongly suggest inclusive/exclusive threshold errors or incorrect ordering of service-band conditions. Exploratory testing may discover a symptom but is unlikely to systematically test each threshold and adjacent value.
Assumption: bonus bands change at 5, 10, and 15 completed years. The actual bonus amounts and whether thresholds use completed years or exact service duration must be specified by payroll policy.

2. BVA test design

Service duration	Purpose	Expected behavior
4 years 11 months	Just below 5-year boundary	Use policy-defined lower band.
Exactly 5 years	First threshold	Use the band defined for 5 years.
5 years 1 month	Just above 5-year boundary	Use policy-defined upper band.
9 years 11 months / 10 years / 10 years 1 month	Around 10-year boundary	Correct lower, boundary, and upper bands.
14 years 11 months / 15 years / 15 years 1 month	Around 15-year boundary	Correct lower, boundary, and upper bands.

3. Equivalence partitions

Input	Partitions	Tests
Years of service	Below first threshold; each bonus band; above highest threshold; invalid negative/future/unknown service	Representative from each band plus boundary tests.
Performance rating	Each allowed rating category; missing/invalid rating	Each rating, invalid value, and band-threshold combinations.
Employment status	Eligible; ineligible; leave/termination status if policy defines it	Check bonus eligibility and exclusion rules.

4. Strategy and justification

- Create a decision table combining service band, performance rating, and eligibility status.
- Test both just below and exactly at each threshold, not just typical employees.
- Use date-based calculations and leap-year/anniversary tests if service is computed from dates.
- Verify rounding, currency precision, and totals against independently calculated expected payouts.
- Use regression tests on each threshold after fixing the conditional logic.

EP gives representative coverage of service and rating classes; BVA targets the known threshold failures. Together they are more systematic and repeatable than exploratory testing alone, though exploratory testing remains useful for discovering unanticipated payroll rules.

7)Laboratory Temperature Controller and 100% MC/DC

1. Safety and failure analysis

Failure to trigger an alarm during a sensor-malfunction sequence is safety-critical. Statement or branch coverage alone may not demonstrate that each Boolean condition independently affects the alarm decision. The precise control algorithm is not included in the assessment, so the following is a conceptual model, not a reconstruction of actual code.

2. Conceptual logic and CFG

Define conditions: H = temperature exceeds high threshold; L = temperature is below low threshold; S = sensor is faulty; A = alarm enabled. A conceptual alarm rule could be D = (H OR L OR S) AND A. The implementation may have different safety requirements and must be reviewed against the approved specification.

Node	Decision / action	Next
1	Read temperature and sensor status	2
2	Sensor faulty? (S)	Yes: 4; No: 3
3	Temperature outside safe range? (H OR L)	Yes: 4; No: 5
4	Evaluate alarm enable / fail-safe behavior	6
5	Normal control: operate heater/cooler as specified	6
6	End / log state	—

A safe design should explicitly specify the response to sensor faults—for example, alarm, safe shutdown, or controlled fallback—rather than silently trusting invalid readings.

3. MC/DC test design

Modified Condition/Decision Coverage (MC/DC) requires each atomic condition to be shown to independently affect the overall decision outcome while other conditions are held appropriately constant. For D = (H OR L OR S) AND A, one illustrative set is:

Test	H	L	S	A	D	Purpose
M1	0	0	0	1	0	Baseline: all hazard conditions false.
M2	1	0	0	1	1	H independently changes D versus M1.
M3	0	1	0	1	1	L independently changes D versus M1.
M4	0	0	1	1	1	S independently changes D versus M1.
M5	1	0	0	0	0	A independently changes D versus M2.

This set illustrates independence for the stated Boolean expression. Actual MC/DC evidence must follow the implemented decision, compiler/coverage tool semantics, and approved safety requirements.

4. Testing strategy

1. Review hazard analysis and requirements for high/low temperature and sensor failure.
2. Construct the CFG and identify every decision and exception path.
3. Develop MC/DC pairs for every atomic condition in each safety-relevant decision.
4. Test sensor open-circuit, short-circuit, stale readings, out-of-range readings, and recovery.

5. Verify heater/cooler interlocks, alarm delivery, logging, and fail-safe behavior on hardware-in-the-loop or an equivalent safe test rig.
6. Repeat tests under timing and fault-injection conditions; review traceability and independent verification.
7. MC/DC improves confidence that each condition can affect a safety decision, but it does not by itself prove absence of hazards. Requirements validation, system-level testing, and safety review remain essential.

8) Travel Booking Promo Codes and Negative Totals

1. Analysis

Testing promo codes independently misses interactions among base price, promotion type, user tier, code stacking order, and eligibility. A percentage discount followed by a fixed discount may produce a different result from the reverse order; stacking may also cause a negative total if validation is absent.

2. Factors and equivalence classes

Factor	Example classes
Base price	Invalid/non-positive; positive below typical threshold; ordinary; high/maximum supported.
Promo type	None; percentage; fixed amount; mutually incompatible promotion; expired/invalid code.
User tier	Free; Basic; Premium; unknown/expired tier.
Stacking	No codes; compatible pair; incompatible pair; duplicate code; too many codes.

3. Decision table

Rule	Price valid	Code valid / compatible	Tier eligible	Expected outcome
R1	Yes	No code	Any	Calculate base total.
R2	No	—	—	Reject invalid price.
R3	Yes	Yes	Yes	Apply promotion according to documented order and caps.
R4	Yes	Yes	No	Reject promotion or calculate without it, as specified.
R5	Yes	Incompatible or duplicate codes	Any	Reject invalid combination or apply specified stacking rule.
R6	Yes	Valid but total would be negative	Any	Prevent negative payable total unless explicitly supported by requirements.

4. Optimized test set

- Base case with no promo code for each tier.
- Each promotion type applied alone at low, typical, and high prices.
- Two compatible codes and two incompatible codes in both application orders.
- Premium-eligible code used by Free and Basic users.
- Expired, duplicated, malformed, and maximum-count promo codes.
- Discount equal to subtotal and discount greater than subtotal.
- Rounding and currency-precision tests after sequential discounts.

Decision table testing ensures important combinations of conditions and actions are represented. Pairwise testing can reduce the broader combination count, but all financially risky stacking and negative-total rules should receive explicit tests.

9)Legacy Migration Tool: Infinite Loop and Cyclomatic Complexity

1. Why statement coverage failed

High statement coverage means statements executed, but it does not prove that loop exit conditions work for every input or that loops terminate. An infinite-loop defect can occur when a counter is not updated, a condition never becomes false, a record repeatedly fails validation without advancing, or nested-loop control uses the wrong variable.

2. CFG and complexity

Model each loop condition as a decision node with a true edge back into the loop and a false edge to the exit. Include error branches, continue/break statements, and nested-loop exits. Cyclomatic complexity $V(G) = E - N + 2P$ estimates independent paths; it guides basis-path selection but does not enumerate all loop iteration counts.

3. Loop boundary tests

Test	Input / condition	Expected result
L1	Zero records	Terminate immediately; no record changes.
L2	One record	Process once and terminate.
L3	Two records	Process both; correct loop advancement.
L4	Maximum supported batch	Complete within performance limit.
L5	Invalid record between valid records	Handle/log invalid record and continue or abort per spec.
L6	Nested loop has zero inner items	Outer loop still advances and terminates.
L7	Failure during record processing	Recover or stop safely; no endless retry.
L8	Large/adversarial dataset	Timeout guard detects non-termination; memory and CPU remain controlled.

4. Strategy

1. Construct CFG and compute cyclomatic complexity.
2. Select independent basis paths and execute each at least once.
3. Test loop counts 0, 1, 2, typical, maximum, and over-limit inputs.
4. Assert that loop counters/iterators advance and every iteration makes progress.
5. Use timeouts, watchdogs, profiling, and logs to detect non-termination.
7. Add fault injection for malformed records and database failures, then run regression tests.

Cyclomatic complexity helps estimate the minimum number of independent paths for basis-path testing. It does not prove termination; loop invariants, progress checks, bounded-input assumptions, and runtime guards are also needed.

10)Food Delivery Tracker: State Transition Testing

1. Analysis

The tracker moves through Ordered, Preparing, Out for Delivery, and Delivered states. A driver cancellation mid-delivery may leave the order stuck because the transition is missing, an event is ignored, or the application does not define recovery and reassignment behavior. EP and BVA focus on input values and may miss sequence-dependent defects.

2. State model

Current state	Event	Next state / action
Ordered	Restaurant accepts	Preparing
Preparing	Food ready and driver assigned	Out for Delivery
Out for Delivery	Successful delivery confirmation	Delivered
Out for Delivery	Driver cancels	Reassignment pending / driver search; notify user
Preparing	Restaurant cancels	Cancelled / refund workflow according to policy
Any active state	Duplicate or stale event	Ignore safely or reconcile; do not regress state.
Delivered	Late cancellation event	Remain Delivered; log/reject invalid transition.

The reassignment-pending state is a proposed design assumption; the actual permitted states must be defined by product requirements.

3. Test scenarios

1. Verify every valid transition in the normal order lifecycle.
2. Attempt invalid transitions, such as Delivered → Preparing, and confirm rejection.
3. Cancel the driver in Out for Delivery and verify the order enters a recoverable state, customer is informed, and reassignment or cancellation follows policy.
4. Send duplicate delivery and cancellation events; ensure idempotent processing.
5. Deliver events out of order or after a timeout and verify reconciliation.

6. Test network loss, app restart, and simultaneous driver/order updates.
7. Verify status, timestamps, notifications, payment/refund, and audit history remain consistent.

Create a state-transition table and transition-coverage matrix. Test each valid transition, important invalid transition, and high-risk event sequence. This directly targets dynamic workflow defects that isolated input partitions may not reveal.

11)MFA Authentication: Error Guessing and Path Analysis

1. Risk analysis

Brute-force vulnerabilities may persist when testing covers normal login success and failure but omits repeated attempts, OTP timing, rate limits, IP/device changes, lockout recovery, and interactions among password, OTP, and location decisions. Security tests must be authorized and conducted in a controlled environment.

2. Hybrid testing strategy

Technique	What to test	Example
Error guessing	Likely abuse and implementation mistakes	Repeated wrong passwords, OTP reuse, expired OTP, rapid resend, account enumeration, reset-flow weaknesses.
Path analysis / CFG	Internal decision paths	Password valid/invalid × OTP valid/invalid/expired × allowed/risky IP × locked/unlocked account.
Boundary testing	Time and attempt thresholds	One attempt below, at, and above configured lockout/rate limits; OTP just before and after expiry.
State/security testing	Sequence and recovery behavior	Lockout, unlock, password reset, session expiry, and concurrent login attempts.
Audit/integration testing	Controls and observability	Rate-limit counters, alerts, logs, and identity-provider integration.

3. Example test matrix

Password	OTP	IP/risk	Expected outcome
Valid	Valid and current	Allowed	Authenticate subject to policy.
Invalid	Any	Any	Reject; increment applicable attempt controls.
Valid	Wrong/expired/reused	Allowed	Reject; no authenticated session.
Valid	Valid	Blocked/high-risk	Apply step-up verification or deny according to policy.
Valid	Valid	Allowed but attempts exceed threshold	Rate-limit, challenge, or lock according to policy.
Valid	Valid	Allowed; concurrent duplicate requests	Prevent replay/session inconsistency.

4. Justification

Error guessing adds experience-based abuse cases, while path analysis ensures that decision branches and combinations are systematically exercised. Use threat modeling, safe rate-limited test accounts, security scanning, logging review, and regression tests. Never conduct brute-force testing against real users or systems without explicit authorization.

12)E-Learning Subscription Expiration and Upgrade

1. Failure analysis

Errors on the exact expiration date suggest ambiguous date boundaries, timezone differences, or incorrect use of inclusive/exclusive comparisons. The requirements must define whether access expires at the start or end of the expiration date and how an upgrade on that date is treated.

2. Equivalence classes for dates and tiers

Dimension	Equivalence classes	Expected behavior
Upgrade date	Before expiry; exactly on expiry; after expiry; invalid/future date outside policy	Apply explicit subscription policy.
Subscription status	Active; expiring today; expired; suspended/cancelled	Grant only entitlements allowed by policy.
Tier	Free; Basic; Premium; invalid tier	Apply corresponding access permissions.
Course type	Free/open; Basic-only; Premium-only; retired/unknown	Grant or deny based on tier and course rules.

3. Boundary-focused test cases

Test	Scenario	Expected result
S1	Upgrade one day before expiry	Upgrade according to policy; preserve eligible remaining time.
S2	Upgrade exactly on expiry date	Apply explicitly documented cutoff and tier transition.
S3	Upgrade one day after expiry	Follow expired-subscription renewal/upgrade rules.
S4	Expiry at midnight and near timezone change	Consistent access decision using authoritative timezone.
S5	Basic user opens Premium-only course	Deny or prompt upgrade.
S6	Premium upgrade payment succeeds but entitlement update fails	Retry/reconcile safely; do not charge twice.
S7	Duplicate upgrade request	Idempotent result and one effective subscription change.

4. Improved strategy

- Define date semantics, timezone, and expiry cutoff precisely.
- Use EP for date ranges, subscription states, tiers, and course classes.
- Apply BVA to expiry timestamps and date boundaries.
- Use a decision table for tier × subscription status × course type × payment status.
- Test daylight-saving changes where relevant, leap days, delayed jobs, and renewal retries.
- Verify permissions through both UI and backend APIs, then automate regression tests.

This strategy addresses the exact boundary and entitlement combinations rather than relying on typical dates alone.

13)Smart-Home Lighting: Branch and Condition Coverage

1. Why statement coverage is insufficient

Executing each statement once does not guarantee that every switch case, Boolean outcome, or sensor condition has been tested. Hardware-software integration failures can arise from sensor thresholds, invalid readings, actuator commands, or combinations of ambient-light and manual-override conditions.

2. Illustrative decision model

Assume a simplified rule: dim lights when ambient light is above a threshold and automatic mode is enabled, unless manual override is active. Let B = ambient light above threshold, A = automatic mode enabled, and M = manual override active. Dimming decision D = B AND A AND NOT M. This is an example only; actual product logic may differ.

Test	B	A	M	Dimming expected
1	0	1	0	No
2	1	1	0	Yes
3	1	0	0	No
4	1	1	1	No
5	0	0	1	No

The tests exercise important condition values and show the independent effects of ambient threshold, automatic mode, and manual override. For full branch/condition coverage, map each decision and switch-case branch in the actual code and ensure every outcome is exercised.

3. Improved white-box strategy

1. Construct a CFG including switch cases, default cases, compound decisions, and error handling.
2. Design tests so each branch outcome executes and each atomic condition evaluates true and false.
3. Test ambient light just below, at, and above the threshold.
4. Test sensor missing, stuck, noisy, or out-of-range values; test manual override and mode changes.
5. Use a hardware-in-the-loop setup to verify real sensor inputs and actuator outputs.
6. Test startup, reset, connectivity loss, and recovery; retain regression tests for hardware/software defects.

Branch coverage reveals unexecuted control outcomes; condition coverage adds evidence about individual Boolean terms. Neither alone guarantees all condition combinations, so MC/DC or pairwise logic testing may be justified for high-risk decisions.

14)Logistics Shipping Cost: Robust BVA

1. Analysis

Acceptance of zero or negative weight suggests missing input validation or tests focused only on typical positive weights. Invalid weights can lead to incorrect charges, corrupted logistics records, or downstream failures. Robust Boundary Value Analysis explicitly includes invalid values outside the permitted range.

2. BVA test design

Assume the minimum billable weight is greater than zero and the system has a configured maximum weight Wmax. The exact minimum increment and Wmax must come from requirements.

Test	Weight input	Purpose / expected result
W1	Negative weight (e.g., −1 unit)	Reject.
W2	Zero	Reject.
W3	Smallest valid positive unit	Accept if other fields are valid.
W4	Just below configured minimum, if one exists	Reject.
W5	Exactly configured minimum	Accept according to policy.
W6	Just below Wmax	Accept.
W7	Exactly Wmax	Accept if inclusive.
W8	Just above Wmax	Reject.
W9	Blank, text, NaN, or unsupported precision	Reject as invalid format.

3. Other input dimensions

- Destination zone: valid zones, unknown zone, and restricted destination.
- Delivery speed: each supported service level and invalid option.
- Combination testing: weight × zone × speed, including expensive/oversized shipment rules.
- Arithmetic testing: rounding, maximum total, overflow, and surcharge calculations.

4. Justification

Robust BVA tests the limits and values just outside them, helping detect off-by-one and missing-validation defects. EP reduces duplication, while decision tables cover shipping rules that depend on combinations of weight, destination, and delivery speed. Negative scenarios are necessary because systems must reject invalid data safely, not merely calculate normal cases.

15)Medical Imaging Parallel Processing and Prime-Sized Data

1. Analysis

Inconsistent results only when dataset size is prime suggest possible indexing, partitioning, chunk-size, remainder-handling, synchronization, or parallel reduction defects. Random testing is weak as the only strategy because it may not deliberately select prime sizes or exercise the relevant loop paths.

2. White-box / basis-path strategy

Construct a CFG for the partitioning and parallel-processing logic. Identify decisions for empty input, partition creation, remainder processing, worker completion, and result merging. Calculate V(G) = E − N + 2P to estimate independent paths and select a basis set.

Test class	Example sizes / condition	Expected result
Empty and minimal	0, 1 item	Defined behavior; no crash or invalid allocation.
Prime sizes	Small and large prime N	Every item processed exactly once; result matches reference.
Composite sizes	Nearby even/composite sizes	Same mathematical correctness as reference.
Partition boundary	N just below, at, and above worker-count × chunk-size	Correct remainder distribution.
Worker boundary	One worker; multiple workers; more workers than items	Consistent result and no lost/duplicated items.
Stress and faults	Large N, worker exception, cancellation	Safe termination and correct error propagation.

3. Example size selection

For a chunk size of 8 and 4 workers, test N = 31 (prime, just below 32), N = 32 (exact multiple), N = 33 (just above), and other prime sizes such as 17 and 101. These are illustrative test inputs, not evidence that the defect has already been reproduced.

4. Validation and justification

1. Compare parallel output against a trusted serial implementation on identical datasets.
2. Assert that each input element is processed exactly once and all output dimensions and values are correct.
3. Test prime and composite sizes, empty/singleton data, exact chunk multiples, and remainders.
4. Exercise independent paths through partitioning, worker scheduling, merge, and error handling.
5. Repeat tests under different worker counts and scheduling orders to expose race conditions.
6. Use deterministic seeds where random data is generated; preserve failing datasets for regression.

Basis-path testing improves structural coverage, while carefully selected prime and partition-boundary sizes target the observed failure pattern. Numerical tolerances must be specified for floating-point algorithms, and deterministic equality should only be required when the algorithm and execution model guarantee it.